

Local LLM inference for document work

DGX Spark · AMD Strix Halo · hosted frontier

One workload, four stacks. September 2026 · Kaushik Ghosh

Why local inference

- Frontier API cost scales per token
- Routine documents don't need frontier models
- Data stays in the building

Two runs of one benchmark suite on the hosted model: about \$47 in API spend.

The workload

12 companies
latest 10-K each

11 financial metrics

121 questions

Ground truth: SEC XBRL
facts

Tolerance: 0.5%

Filings: 45K–124K tokens

Revenue, net income, EPS, total assets, long-term debt, shares outstanding, ... pulled from the filing text.

The two machines

NVIDIA DGX Spark

GB10 · 128 GB · Linux

llama.cpp built from source, CUDA

AMD "lunch box" (Strix Halo)

Ryzen AI Max+ 395 · Radeon 8060S · 128 GB · Windows

Locked down → LM Studio (Vulkan, ROCm)

Same model file on both: gpt-oss-120b, MXFP4.

DGX Spark, 26 min	AMD box, Vulkan, 77 min	AMD box, ROCm, 71 min
gpt-oss-120b MXFP4 GGUF weights	gpt-oss-120b MXFP4 GGUF weights	gpt-oss-120b MXFP4 GGUF weights
llama-server pinned llama.cpp build	LM Studio bundled llama.cpp 2.37	LM Studio bundled llama.cpp 2.37
ggml-cuda engine backend	ggml-vulkan engine backend	ggml-hip engine backend
CUDA + cuBLAS NVIDIA compute API	Vulkan cross-vendor API	ROCm / HIP AMD compute API
DGX OS Ubuntu, ARM64	Windows 11 x86-64	Windows 11 x86-64
GB10 Blackwell tensor cores, 128 GB	Radeon 8060S RDNA 3.5, 128 GB	Radeon 8060S RDNA 3.5, 128 GB

Prefill vs decode

Prefill — reading the prompt

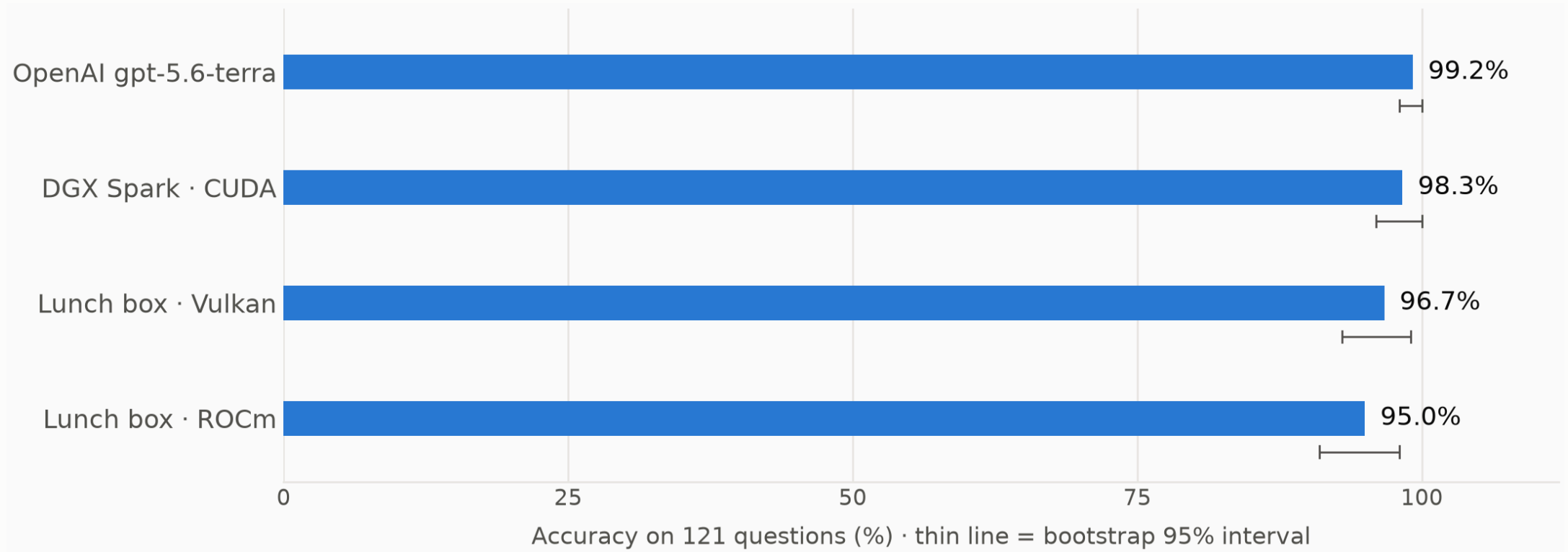
- Happens once per document
- Compute-bound
- Where the two boxes differ

Decode — writing the answer

- Happens for every output token
- Memory-bandwidth-bound
- Nearly identical on both boxes

Prompt cache: pay prefill once per document; follow-up questions reuse it.

Accuracy



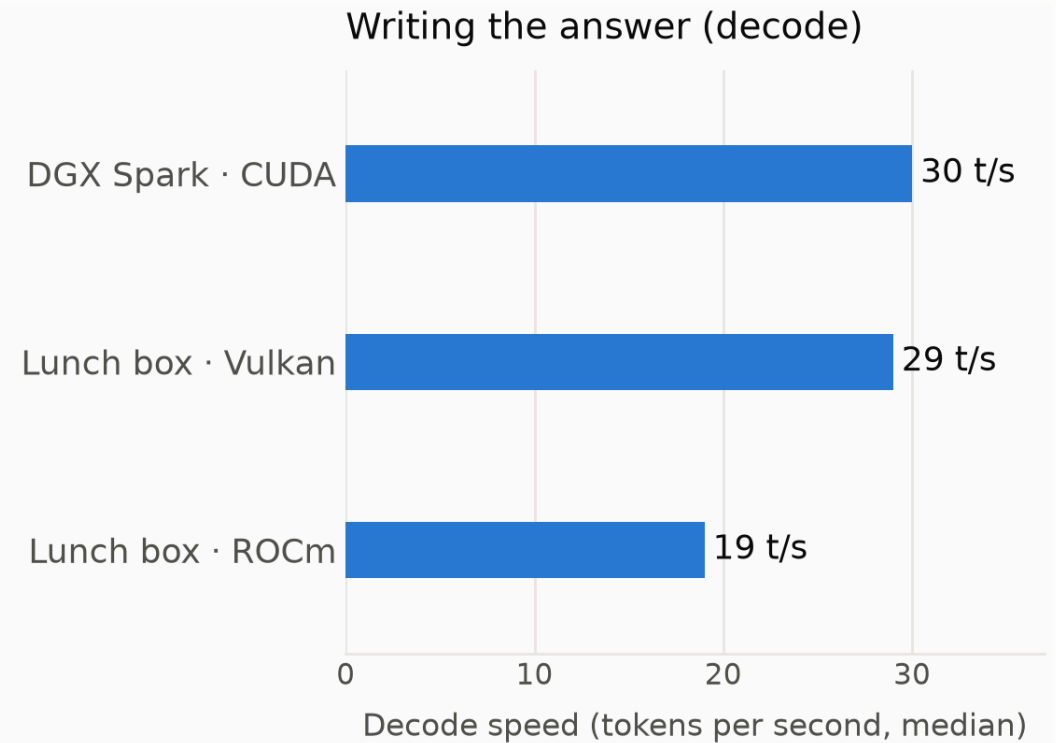
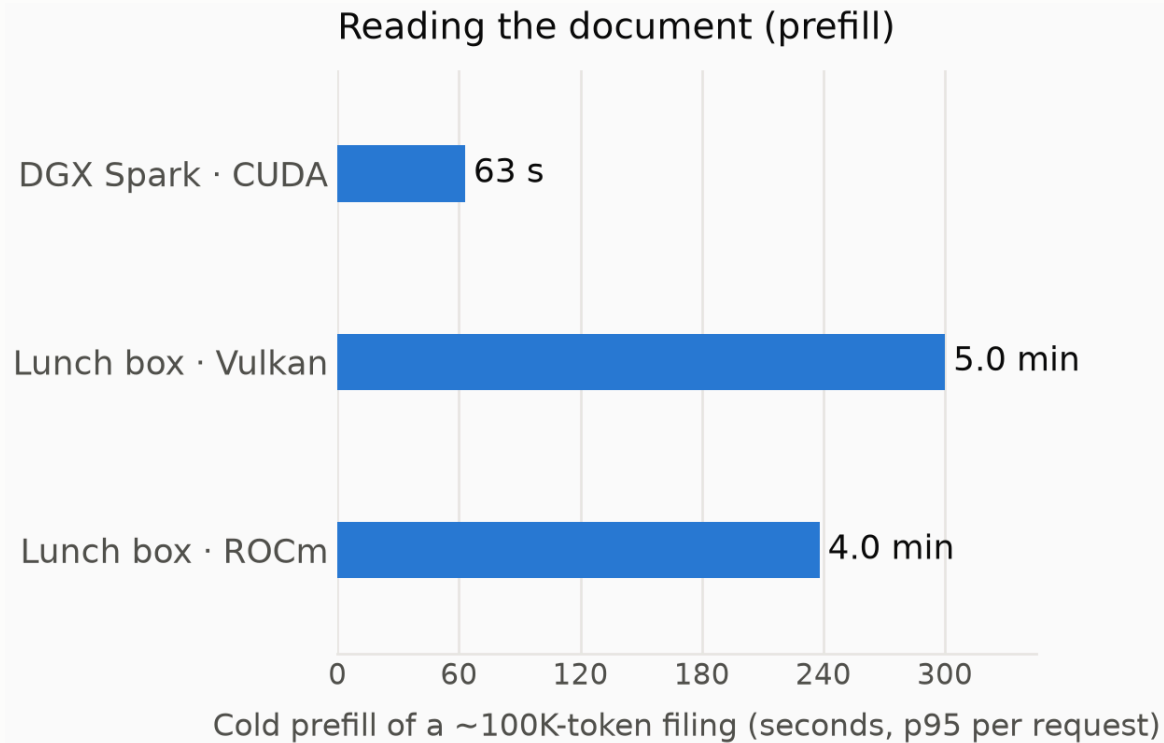
Single run per stack. The intervals overlap: not distinguishable at this sample size.

Results

Stack	Accuracy (121)	Fits in ctx (104)	Decode	Cold prefill ¹	Whole run	Cost / run
OpenAI gpt-5.6-terra	99.2%	103/104	—	—	20 min	~\$25
DGX Spark · CUDA	98.3%	103/104	30 t/s	40–60 s	26 min	\$0
Lunch box · Vulkan	96.7%	103/104	29 t/s	4–8 min	77 min	\$0
Lunch box · ROCm	95.0%	101/104	19 t/s	4–8 min	72 min	\$0
DGX Spark · gpt-oss-20b	87.6%	90.4%	45 t/s	40–60 s	19 min	\$0

¹ first question on a new ~100K-token filing. Local: gpt-oss-120b MXFP4 unless noted; \$0 = no per-token cost (hardware, power, time excluded). Hosted cost estimated from tokens at list price.

Where the time goes



Same decode speed. Prefill is 4–5x slower on the lunch box, and it is the whole wall-clock gap.

What it means

- **Background extraction** → go, on either box
- **User-facing flows** → design around cold prefill
minutes on the lunch box · about a minute on the Spark
- **Cost** → ~\$25 per suite run on the API vs \$0
marginal locally

Caveats

- One run per stack; confidence intervals overlap
- Memorisation control not yet run
public filings are in every model's training data
- Lunch box ran LM Studio's llama.cpp, not our pinned build
- Serial requests only; nothing measured under concurrency

Next

- Run the no-document control
- Concurrency sweep on both boxes
- Linux + native llama-server on a lunch box
- A second use case

github.com/skiingfalcon/llama-cpp-spark → docs/index.md